

Question 1

Lord Binarius, Magician of Data-Structures has lead you into a room only for you to be trapped! The entrance door closes behind you and two exit doors lay ahead. For your crimes against chirality, Lord Binarius obscures the final exit of his dungeon by repeatedly creating new identical rooms and connecting their entrance doors to exit doors of previously created rooms. Lord Binarius always creates rooms such that they connect to rooms that have existed for the longest amount of time and that have at least one door that is not already connected to another door. When connecting two rooms, Lord Binarius prioritises the left exit door first, connecting rooms via the right exit door second. The final room that Lord Binarius places will be the n^{th} room and shall hold the exit to his dungeon.

1.1 Lord Binarius reluctantly agrees to your safe return only if you find the exit at record speed. Design a *divide-and-conquer* algorithm that finds the exit in $O(\log n \log n)$ time.

The data structure that this set of rooms form is a binary tree. Since nodes are connected to the oldest available node, then nodes are added in FIFO queue order similarly to level order traversal. Therefore, the binary tree is a complete binary tree, with nodes filling the latest level from left to right. The final node will be in this final level. On this final level, all leaf nodes to the left of and including final node will have a distance, h from the root and all leaf nodes to the right will have a distance $h - 1$. Therefore, binary search can be used on this level to find the final node.

Find the max height, h of the tree by traversing the left side of the tree and counting rooms until the left-most leaf node is reached. Return back to the root node and traverse to the left-most leaf node of the right sub-tree (the middle node of the final level). Since the tree is complete, then the heights of sub-trees can only differ by at most 1, and so the height of the right sub-tree can be only either $h - 1$ or $h - 2$. If the height of the right sub-tree is $h - 1$ then the final node must be in the right sub-tree, alternatively, if the height is $h - 2$ then the final node must be in the left sub-tree. Recursively search the sub-tree that includes the final node until the final tree reached is a leaf node. This is the final node. Finding the sub-tree that contains the exit requires traversing to a leaf node to find the height, so $O(\log n)$, but this operation is performed for each level of the tree until a leaf node is reached, so a total of $\log n$ levels. So in total, the algorithm is $O(\log n \log n)$.

[6 marks]

- [1 mark] Identifies data structure as a complete binary tree
- [2 mark] Employs a divide-and-conquer strategy. Binary search accepted.
- [3 mark] Algorithm is correct and is in $O(\log n \log n)$

1.2 Lord Binarius cannot believe that you managed to solve the dungeon in $O(\log(n) \log(n))$ time, he now wants you to prove that you really solved it by asking you to solve 1.1 in pseudo-code.

```
GetHeight(node):
    if node == Null
        return -1
    if node->left == Null
        return 0
```

```

    return GetHeight(node→left) + 1
end of function

FindExit(currentNode, currentHeight):
    if node→left == Null and node→right == Null
        return current node→value

    RHeight ← GetHeight(currentNode→right)

    if rightHeight == currentHeight - 1
        return FindExit(currentNode→right, currentHeight - 1)
    else
        return FindExit(currentNode→left, currentHeight - 1)
end of function

For a given root entrance the exit room number is found by

do GetHeight(root node)→ Height
then do FindExit(root,Height) → ExitRoom

```

[4 marks]

- [1 mark] Correctly Outlines GetHeight() or similar function
- [2 marks] Correct logic within searching algorithm/function
- [1 mark] Pseudocode matches the algorithm explained in 1.1

1.3 Define a recurrence relation $T(n)$ for the above algorithm, and prove the time complexity of this algorithm is $O(\log n \log n)$. However, you will find that the lecture provided Master theorem will not be able to solve this question. Therefore, prove that the lecture provided Master theorem cannot solve this recurrence relation and use the provided definition of the Master theorem to solve this.

Theorem*Suppose*

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

where $f(n) \in \Theta(n^c(\log n)^k)$.

Let $d = \log_b(a)$. Then:

Case 1: If $c < d$ then $T(n) = \Theta(n^d)$

Case 2: If $c = d$ then $T(n) = \Theta(n^c(\log n)^{k+1})$

Case 3: If $c > d$ then $T(n) = \Theta(f(n))$

Before we do the recursive function "FindExit", we do the "GetHeight" function first. It costs $O(\log n)$ time because it divides the binary tree into 2 sub-tree and always does left sub-tree for each time. Then the recurrence relation of "FindExit" function is $T(n) = T(n/2) + O(\log n)$. But the lecture provided Master Theorem cannot solve this recurrence relation.

Proof of why the lecture provided Master Theorem cannot solve the recurrence relation of the function "FindExit", $T(n) = T(n/2) + O(\log n)$:

For all cases we can see that $a = 1$, $b = 2$, and $f(n) = \log n = \Theta(\log n)$

Case 1:

Since $n^{\log_2 1 - \epsilon} = n^{0 - \epsilon}$ which for any value of $\epsilon > 0$ does not equal $f(n)$, therefore case 1 fails

Case 2:

Since $\Theta(n^{\log_2 1}) = \Theta(n^0) = \Theta(1)$, which is not equal to $f(n)$. So case 2 fails.

Case 3:

Since $\Omega(n^{\log_2 1 + \epsilon}) = \Omega(n^{0 + \epsilon}) \leq \Omega(n^{0.1})$ and $\log n! = \Omega(n^{0.1})$, for $\epsilon = 0.1 > 0$, and $af(n/b) = f(n/2) = \log n/2 = \log n - 1$, but we can't find some $c < 1$, and n_0 , for all $n \geq n_0$, $\log n - 1 \leq c \log n$. Because those two functions are not "parallel", and $\log n - 1$ increases faster than $c \log n$ after the point of intersection. So case 3 fails.

Instead we can use the provided Master Theorem to obtain the following,

In this algorithm, the $a = 1$, $b = 2$, $c = 0$, $k = 1$, and $d = \log_b(a) = \log_2 1 = 0$. Therefore, $c = d = 0$ which satisfies Case 2, then $T(n) = \Theta(n^c (\log n)^{k+1}) = O(\log n \log n)$. Therefore for the whole algorithm, the time complexity would be $O(\log n + (\log n \log n)) = O(\log n \log n)$.

[5 marks]

- [1 mark] Identifies the correct recurrence relation $T(n)$.
- [2 mark] Correctly proves why the lecture provided Master Theorem cannot solve this recurrence relation
- [2 mark] Proof is correct and utilises master theorem recurrence relation (Proof also has to show the time complexities of all helper functions used, such as the GetHeight function).

1.4 Finally, after many dangerous twists and turns you narrowly escape the dungeon, but wait, a frenzied scream echos ahead as Lord Binarius's illusory magic descends on you once more. For your hubris, Lord Binarius traps you in a new dungeon with each room now containing k exit doors. Lord Binarius creates and connects rooms with a similar strategy to **Question 1**, instead prioritising the left most exit door when connecting rooms. Lord Binarius places the exit in the n^{th} and final room. Was your escape from the last dungeon dumb luck or will you be able to escape again this time in $O(\log k \log n \log n)$ time? Design a *divide-and-conquer* algorithm that finds the exit in $O(\log k \log n \log n)$ time.

The new dungeon forms a complete k -ary tree. If we can locate the sub-tree that contains the exit node, like in **1.2**, then we can recursively search this sub-tree for the exit. First find the height, h of the tree and then find the height of the $\lfloor k/2^{\text{th}} \rfloor$ sub-tree. If the height of this sub-tree is $h - 1$ then the index of the sub-tree that contains the exit node is $\lfloor k/2 \rfloor$ or greater. Otherwise if the height is $h - 2$, the index of the sub-tree that contains the exit node will be lesser. By keeping track of the range of possible indices of sub-trees the exit node can be contained in, we can use binary search to find the sub-tree that contains the exit node. Use binary search to recursively find the next sub-tree and then recursively search the sub-tree for the exit node until the final tree reached is a leaf node. This will be the exit. Performing binary search on k items is $O(\log k)$, but querying a mid point by finding the height of a sub-tree takes $O(\log n)$. So finding the sub-tree that contains the exit node

is $O(\log k \log n)$. This operation is performed for each level of the binary tree, in total $\log n$ levels. So in total, the algorithm is $O(\log k \log n \log n)$.

[5 marks]

- [2 mark] Employs a divide-and-conquer strategy. Binary search accepted.
- [3 mark] Algorithm is correct and is $O(\log k \log n \log n)$

Group name: Ext Group

Please list the group members and their contribution. Example:

Name	zID	Contributions
A	z5406283	Q1 intro, 1.1, 1.4
B	z5280792	Marking rubrics and Editing
C	z5275224	1.3
D	z5361758	1.2 and 1.3

Question 1 inspired by -

<https://leetcode.com/problems/count-complete-tree-nodes/description/>